

ZPR PWr – Zintegrowany Program Rozwoju Politechniki Wrocławskiej

Web System Programming

W01 – Introduction

Syllabus

- Overview introduction
 - HTML 5
 - CSS3
 - JavaScript
 - Browsers and HTML5, CSS3 and JS compatibility
 - Validators
 - URL, URI
 - HTTP protocol - GET and POST requests
 - `curl` console tool
 - Browser's tools
 - Multi-tier application architecture
 - Client and server side scripts/programs
- HTML5
 - Simple document structure – tags `<html>`, `<head>`, `<title>`, `<body>`, `<p>`
 - HTML validators
 - Headers: tags `<h1>` do `<h6>`
 - Linking: `<a>` tag
 - Pictures: `<img>` tag

Introduction

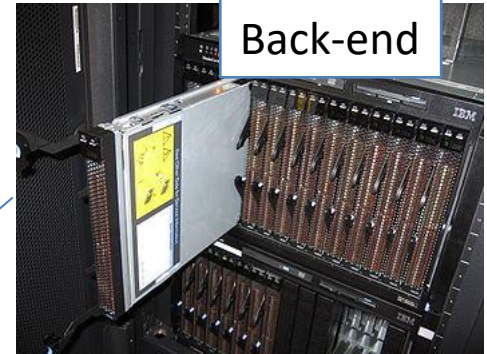
- A **feature** of programming **web application** is their **portability**, especially when it comes to the ability to **run** them on **multiple devices with Internet access**.
- The feature that **distinguishes web applications** from **websites** is **interactivity** and **very often communication** with the server in order to perform certain functionality.
 - The **web application** may also **not communicate** with the server at all, or cache previously downloaded data.
- The web application is divided into two parts:
 - **Front-end**
 - The **client-side** part of the application, most often browsers
 - **Back-end**
 - The **server-side** part of the application

Client
Front-end



Internet

Server
Back-end



[To zdjęcie](#), autor: Nieznany autor, licencja: [CC BY-SA](#)

Client-side/server-side techniques

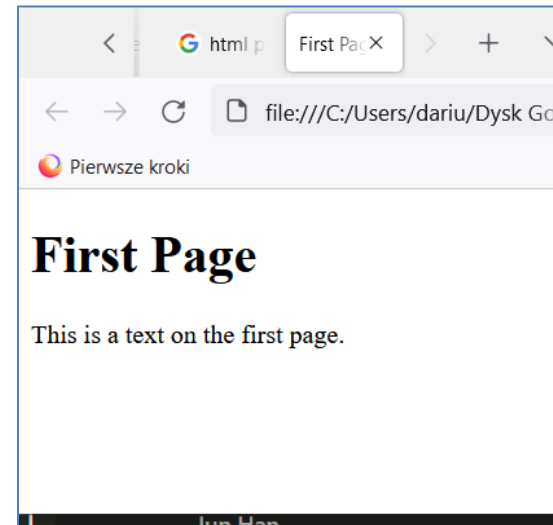
- **Client-side** programming techniques are used to **build websites** and applications that run on **the client**:
 - HTML5, CSS3, JavaScript, Ajax
 - Frameworks JS/Typescript (all are finally translated to JS/CSS):
 - Bootstrap, jQuery etc. to support the user interface.
 - Angular/React(this course)/Vue for building SPA (Single Page Application) applications.
 - and others
- **Server-side** programming techniques - **applications** that respond to **browser requests (client side)**.
 - PHP language
 - Java language
 - Framework Spring (this course)
 - other
 - JavaScript: Node.js
 - C #: ASP .Net
 - Python, Ruby et al.

- HTML (*HyperText Markup Language*) is a **markup language**. It was created to define the **content** and **structure** of web pages (documents) in a portable way.

- The presentation of the content **may depend** on the **device** (computer, smartphones, tablets, big screens) and software, e.g. **browser**.
 - However, **most software (browsers)** try to **follow** the **recommendations** of the standard.

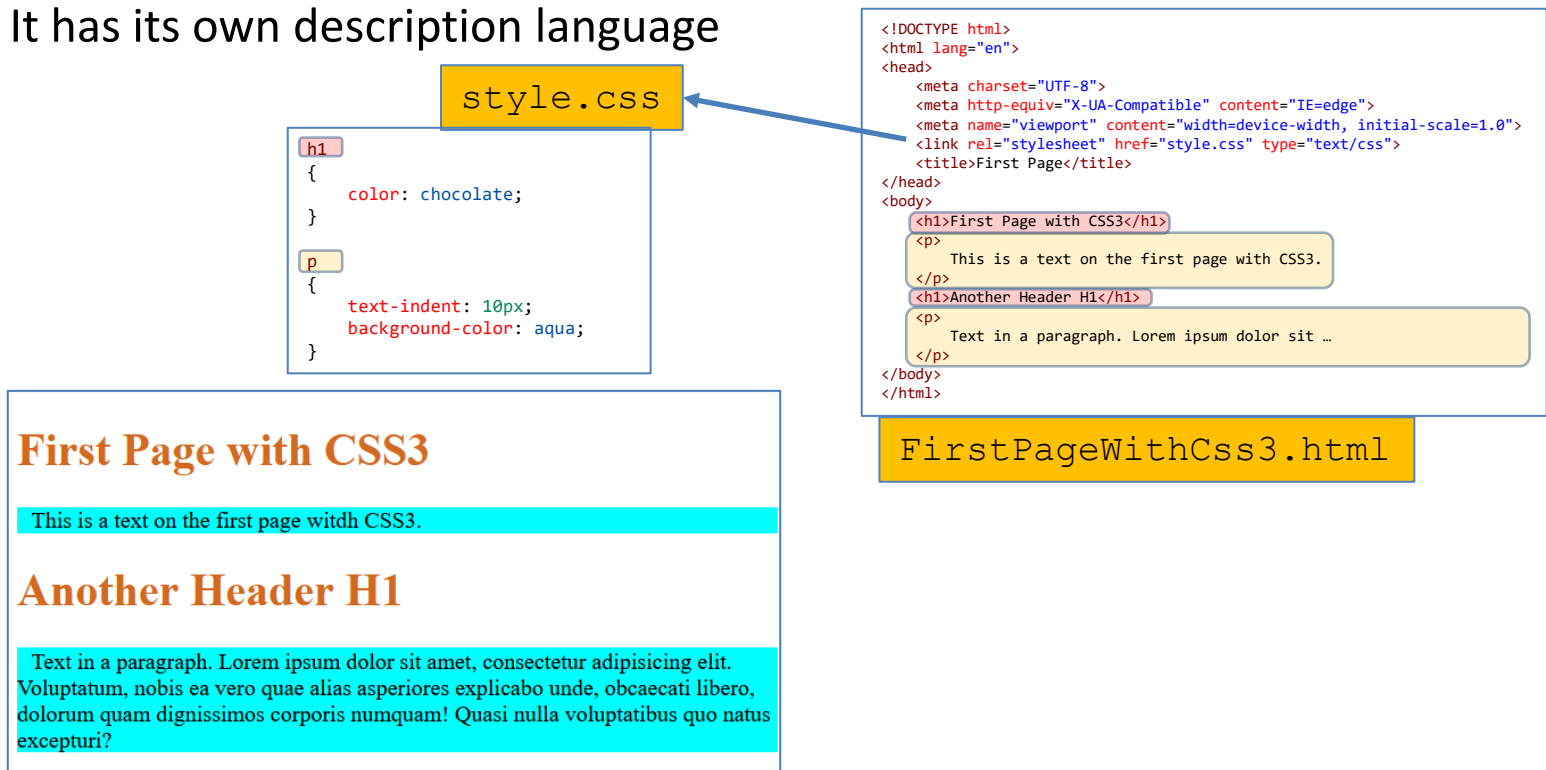
- HTML5 is the **current standard**, but still **under development**.
 - HTML6 etc. is not foreseen.
- There is a more restrictive XHTML (*Extensible HyperText Markup Language*) standard that is based on XML (*eXtensible Markup Language*)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>First Page</title>
</head>
<body>
  <h1>First Page</h1>
  <p>
    This is a text on the first page.
  </p>
</body>
</html>
```



CSS3 - Cascading Style Sheets

- Theoretically, in HTML5, the content and structure of a document **can be mixed** with information about its presentation, but this is **not advisable**
 - It makes it impossible to easily change the graphic design, etc.
- CSS3 - **Cascading Style Sheets version 3** allow to **separate** information about the **content presentation** into a **separate file fragment** or even a **separate file**.
 - It has its own description language



CSS3

- **Cascading Style Sheets (CSS)** allows you to define the **presentation** or **styling** of elements on the page (e.g. fonts, spacing, sizes, colors, alignment etc.).
- Because CSS separates style from content and structure, you can **easily change** the look and feel of **all or part** of a website by changing the style in one file.
- CSS had versions 1 to 3. CSS3 is also subject to development. Often when a CSS abbreviation is used, it implies CSS3.

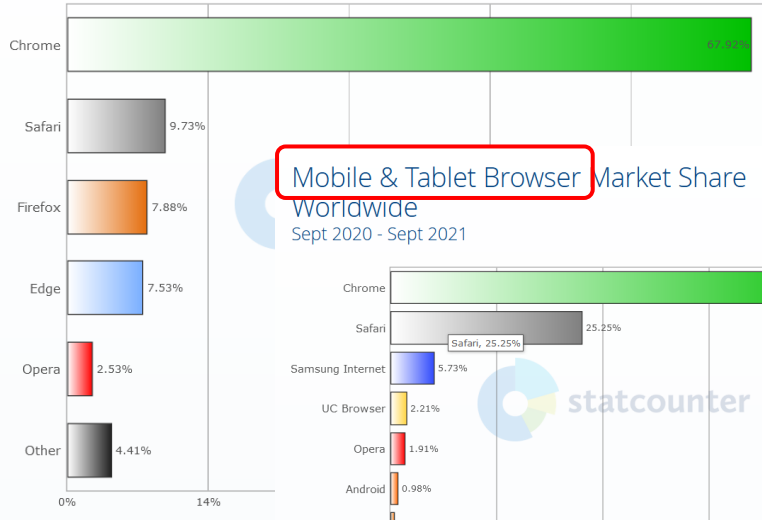
JavaScript language

- JavaScript - scripting language (used to be only for web pages)
 - to **manipulate** the **content** of web pages
 - allows you to create **dynamic pages** that, for example, **react to events** such as mouse movement, time change, etc.
 - It also allows programming from server-side: **Node.js**
- Creators: Netscape (JavaScript) first, then Microsoft (JScript)
- JavaScript standardization in ECMA International (formerly European Computer Manufacturers Association) as ECMAScript.
- ECMAScript 5 (ES5) - a very popular version
- ECMAScript 6 (ES6) - standard version with classes and objects.
 - ES6 == ES2015
 - Current version of ES2021, asynchronous programming etc.
- Programs written in JavaScript can run in web browsers on many devices.
 - You can also **disable** JavaScript **in** your **browser**.
- Theoretically, there may be other client-side scripting languages executed on browser, but only theoretically:
 - VBScript for Internet Explorer up to version 10, then no longer supported.

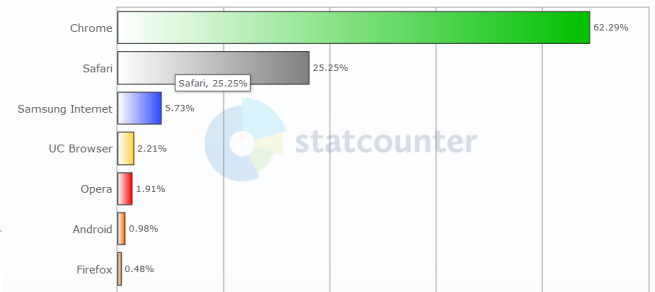
Web browsers

- Browsers have **similar capabilities**, but each can interpret and display pages **differently**
 - There is **no standard** that software producers must adhere to when creating web browsers.
 - **Support** for HTML5, CSS3 and JavaScript capabilities etc. is **browser dependent**.
 - User **preferences** may depend on browser characteristics **other than** HTML5 "compatibility,"
- Data from gs.statcounter.com

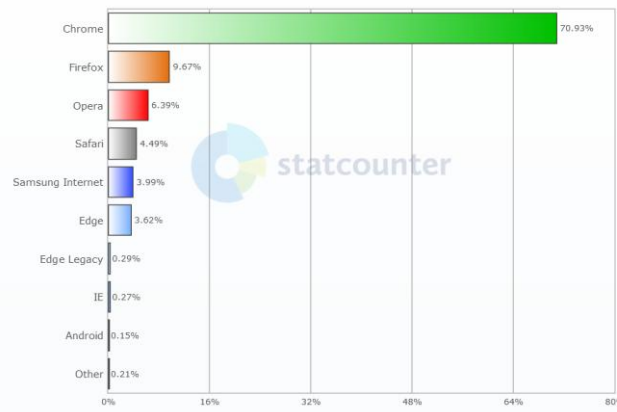
Desktop Browser Market Share Worldwide
Sept 2020 - Sept 2021



Mobile & Tablet Browser Market Share Worldwide
Sept 2020 - Sept 2021

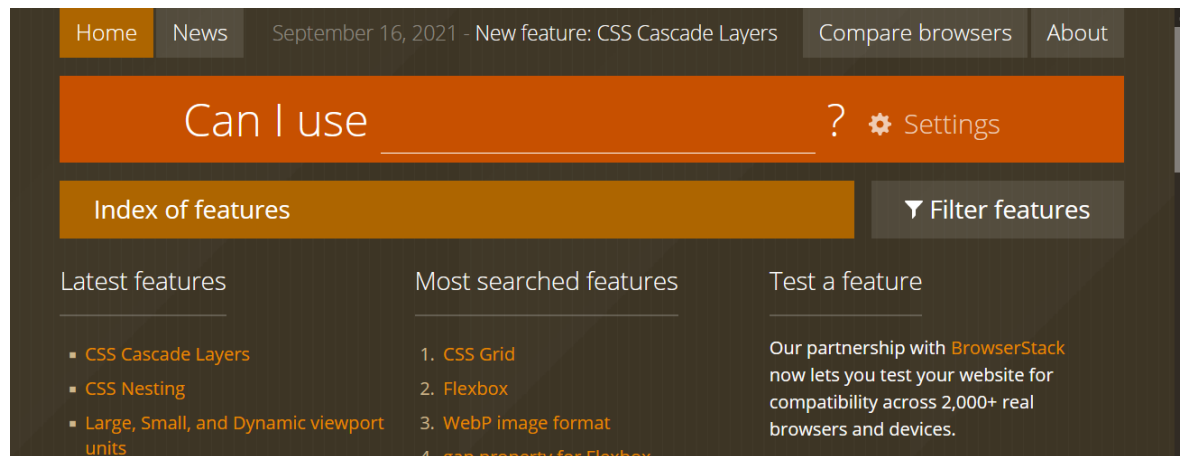


Browser Market Share Poland
Sept 2020 - Sept 2021



HTML5 and CSS3 features available: caniuse.com

- Checking the availability of a given function HTML5 i CSS3:
 - <http://caniuse.com/>

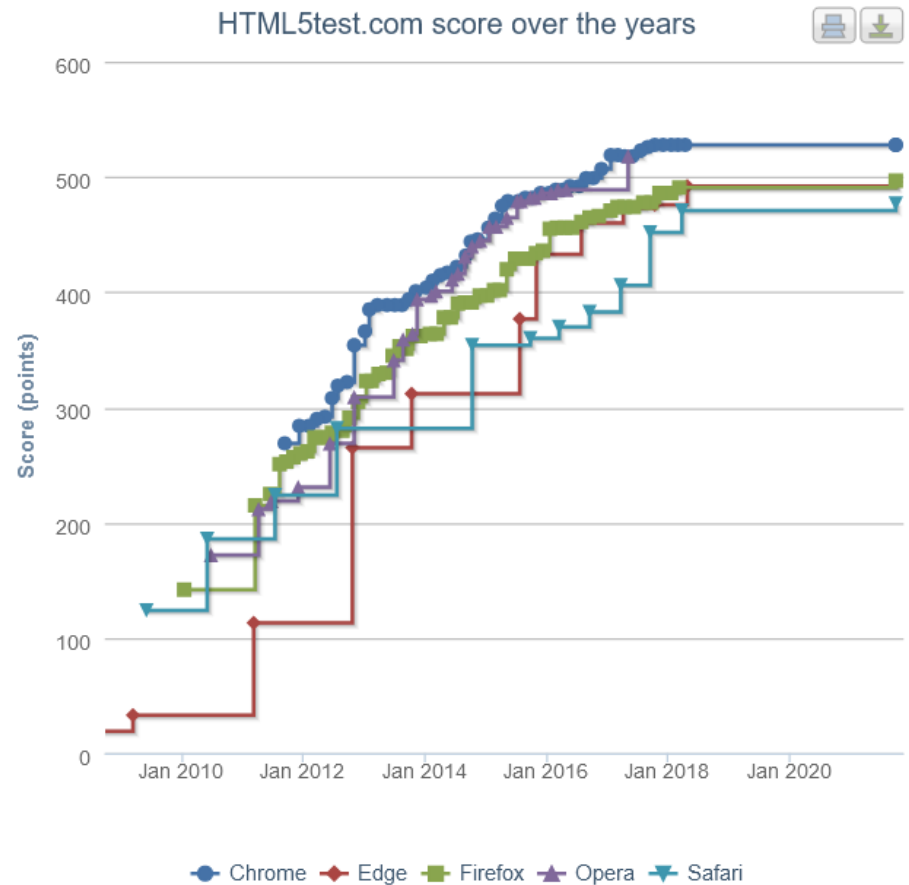
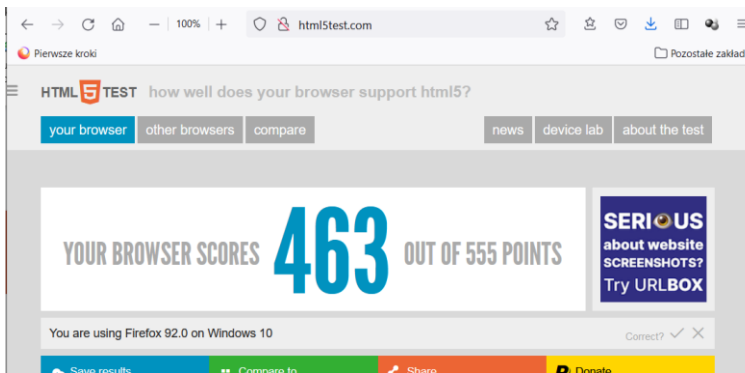


Browser Compatibility Assessment : html5test.com

- Browser Compatibility Assessment:

– <http://html5test.com/>

TIMELINE



Code validators for HTML5, CSS3 and JavaScript

- Code validators check the **syntax** of HTML5, CSS3, and JavaScript code to let browsers parse the documents correctly.
 - Browsers, in the event of errors, try to **guess** what the creator of the website was about and correct it "on the fly", so when viewing the effect in the browser, the error will **not always be noticed**.
- Below are the most important validators.
- Another interesting validator and tools for web developers:
 - <https://www.freeformatter.com/>

Language	Validators
HTML5	http://validator.w3.org/ https://html5.validator.nu/
CSS3	http://jigsaw.w3.org/css-validator/ http://www.css-validator.org/
JavaScript	https://beautifytools.com/javascript-validator.php https://www.jshint.com/

WWW, HTML, HTTP

- How **packet transfer** works - Course "Computer networks,,
 - Packet switching
 - TCP/IP protocol
 - Network routing
- The World Wide Web allows you to use web applications and download multimedia.
- Tim Berners-Lee of CERN (European Organization for Nuclear Research) created:
 - **HTML language** - HyperText Markup Language.
 - **HTTP** (Hypertext Transfer Protocol) - for sending information over the Internet
 - There were/are also other protocols, e.g. gopher, with greater restrictions as to the structure of the document.
 - **Proposed URL** (Uniform Resource Locator) - a unique resource locator that specifies the address (location) of the website (and not only) displayed in the browser window
- The URL for websites starts with `http://` lub `https://`

HTTPS

- HTTPS - Hypertext Transfer Protocol **Secure**.
- It is the standard for transmitting encrypted data over the Internet.
- It combines HTTP with **Secure Sockets Layer (SSL)** and a cryptographic system - **Transport Layer Security (TLS)** - to secure communication and confidential information on the Internet.
- A website URL starting with `https://` is the current standard due to the confidentiality of the content of requests and responses.

WWW basics - hyperlinks

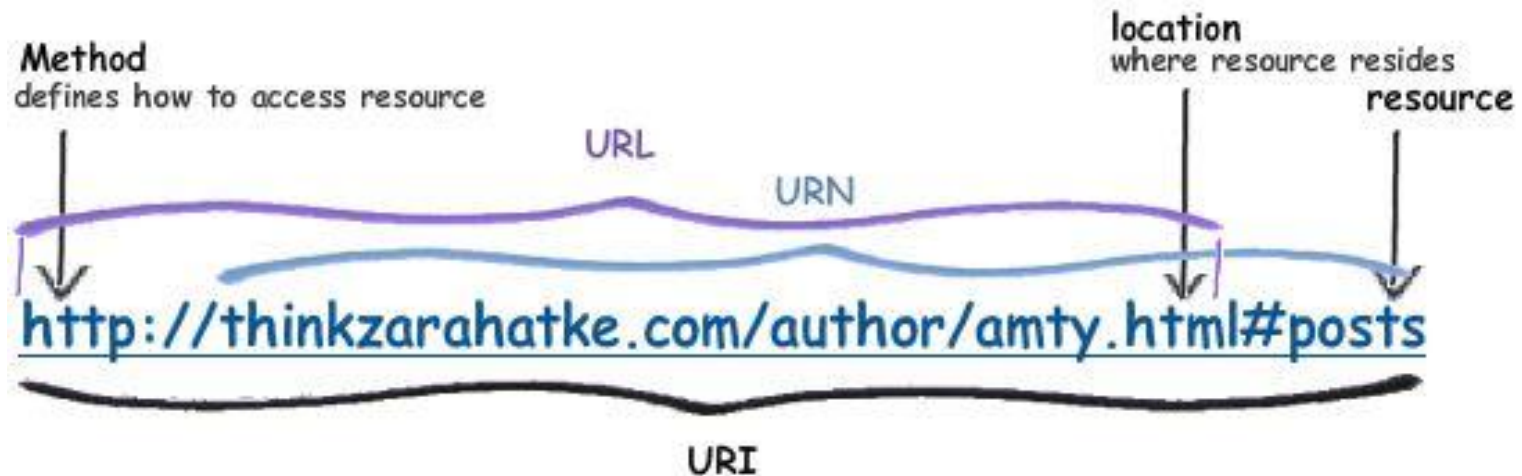
- In the simplest case, a website is an HTML (HyperText Markup Language) file with the .html or .htm extension, which describes the **content** and **structure** of the document in the web browser.
- HTML documents usually contain **hyperlinks** that point to **another resource** (for example, when clicked, they load another web document).
- Hyperlinks can lead to text/document, image, video, etc.
- When a user **clicks** a hyperlink, the **browser communicates** with the target web server which **locates** the requested resource (e.g. a website) and **sends** it to the user's browser.
- The user can also **type** the address of the resource in the **browser's address bar** and press Enter to **download** it (and display it if it is a web page).

Hyperlinks - cont.

- In the case of a hyperlink URL in the form of `mailto: e-mail address`, clicking on the link starts the **default e-mail client** and opens the window of the message **addressed to** the indicated recipient.
- There are several different types of hyperlinks (information will appear during the lecture)
- In the case of a hyperlink **to a file** that the browser is **unable** to view, it prepares to **download** it and asks the user where to **save it** (or saves it to the default folder).

URL/URI/URN

- URI (**Uniform Resource Identifier**) identifies the **resource** on the Internet along **with the method** (protocol) of access.
- URL (**Uniform Resource Locator**) – an URI, without specifying a specific place (the part after hash ,#' character) in the resource (the entire resource)
 - WWW pages are a resource that starts with `http://` (or `https://`)
- URN (**Uniform Resource Name**) - an URI that indicates the name of the resource (no protocol).
- There is also a place for **login** and **password** in URI (open code, so rarely used)

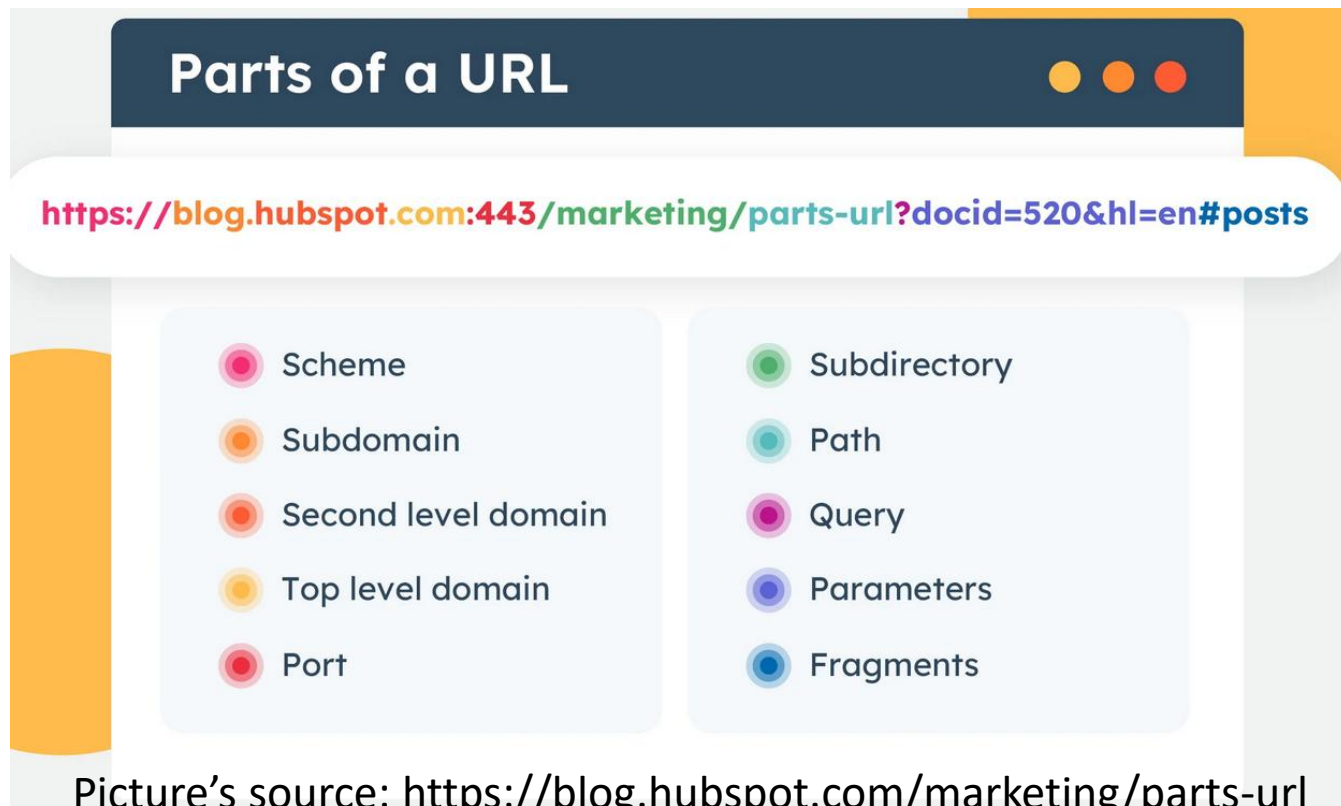


Picture's source:

<https://stackoverflow.com/questions/42534419/examples-of-uri-url-and-urn>

URL - components

- Web servers analyze sent URL/URI and provide the requested resource (if it exists and we have permissions to it)
- Examples of WWW servers (and not only WWW):
 - Apache HTTP Server
 - Microsoft Internet Information Services
- Address components: upper figure.
- There may also be query items in the URL.



Operation analysis

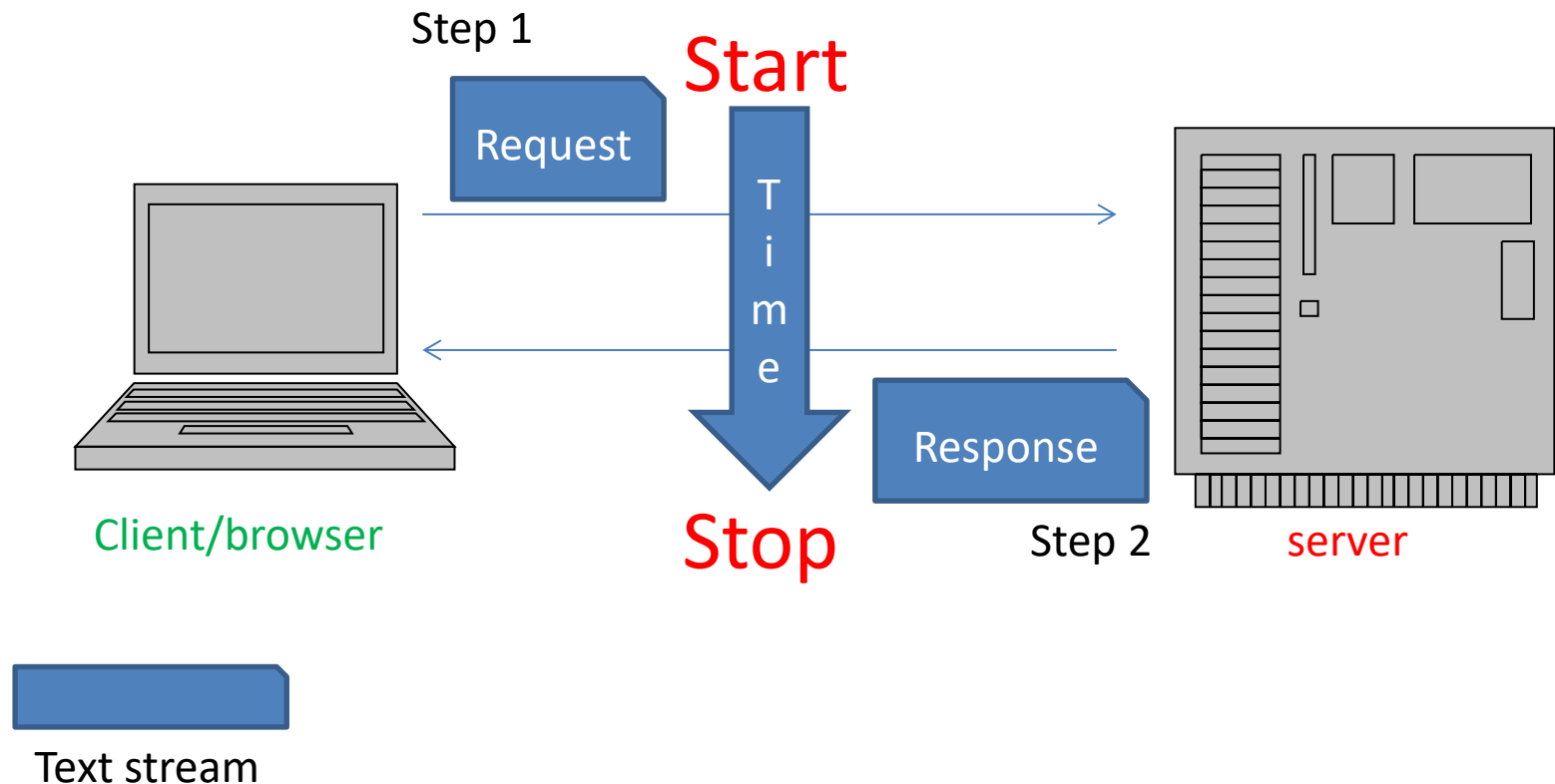
- Sample URL: <https://www.dblog.digad.pl/artykuly/20/url.html>
- The fragment `https://` indicates that HyperText Transfer **Protocol** Secure (HTTPS) should be applied to obtain the resource..
- The fragment www.dblog.digad.pl is the **host name** (hosts the resource), i.e. the name of the computer with the web server on which the resource is located.
 - The host name www.dblog.digad.pl is translated into an IP (Internet Protocol) address - a 4-byte number that uniquely identifies the server on the Internet. This is done by DNS (Domain Name System) servers that manage a database of host names and corresponding IP addresses.
- The fragment `/artykuly/20/` stands for the resource location **path** on the host
- The fragment `url.html` - the specific **resource** on this path
- The resource path **does not have to match** the actual tree of folders and files on the server.
 - The simplest solution to mapping a URL to a path in the server folder tree :

sun10.pwr.edu.pl/~koniecz/mac/macierz.html URN

sun10.pwr.edu.pl
/users/staff/koniecz/wwwroot/mac/macierz.html Path on sun10

HTTP stateless protocol

- Stateless, no history, no request sequence remembering
 - This is a **protocol** feature
- In practice, if you need to create a session from subsequent requests, the browser and the server must cooperate by sending relevant information, most often in the headers (cookies).



Request/response structure in HTTP

- The **request** and **response** are a **text stream** of lines separated by a newline character
 - Can be treated like transferred text files (.TXT)
- The request and the response have **the same structure**:
 - **One line** of request and response
 - Subsequent lines **up to the first empty** one - **header lines**
 - **Blank line** (separator)
 - **The rest of the lines** (can also be empty) - the **body** of the request/response
- There are many **tools** for **observing** requests and responses. The simplest and most often at hand are:
 - Curl command/console program
 - In any sensible browser - „Developer Tools“

Command: curl

- curl is a tool for transferring data to or from a server over the HTTP(S) protocol.
- Without options, print the content of the resulting HTML stream.
- The -h option lists all options, their abbreviations, and what additional parameters are required
- Below the effect of calling the command:
 - curl -h
 - curl <http://www.wp.pl/>
- The last call indicates that there is (theoretically) an error page:
 - 303 Moved Permanently
 - So the page has been permanently transferred (to a different address)
- However, there is no information to which other address the page was moved to. To check this, select the option that also shows the answer line and response headers.

Wiersz polecenia

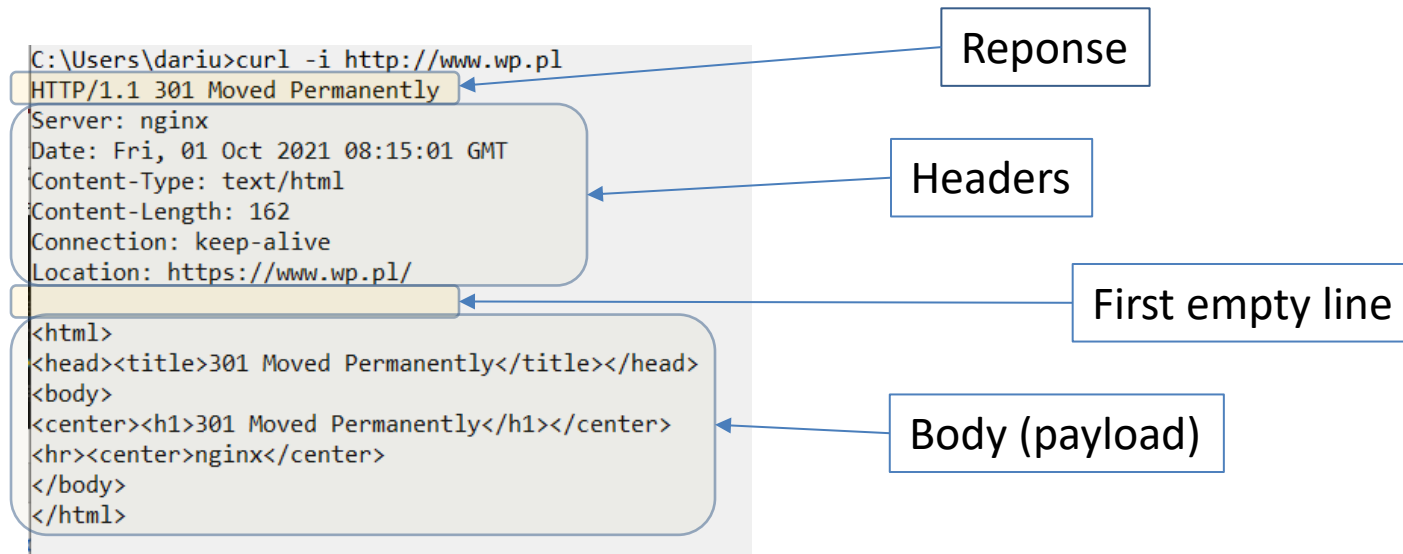
```
C:\Users\darius>curl
curl: try 'curl --help' for more information

C:\Users\darius>curl -h
Usage: curl [options...] <url>
  --abstract-unix-socket <path> Connect via abstract Unix domain socket
  --anyauth             Pick any authentication method
  -a, --append          Append to target file when uploading
  --basic              Use HTTP Basic Authentication
  --cacert <CA certificate> CA certificate to verify peer against
  --capath <dir>       CA directory to verify peer against
  -E, --cert <certificate[:password]> Client certificate file and password
  --cert-status        Verify the status of the server certificate
  --cert-type <type>   Certificate file type (DER/PEM/ENG)
  --ciphers <list of ciphers> SSL ciphers to use
  --compressed         Request compressed response
  -K, --config <file>  Read config from a file
  --connect-timeout <seconds> Maximum time allowed for connection
  --connect-to <HOST1:PORT1:HOST2:PORT2> Connect to host
  -C, --continue-at <offset> Resume transfer offset
```

```
C:\Users\darius>curl http://www.wp.pl
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

Command `curl` – interesting options

- The option `-i`, `--include` Include protocol **response headers** in the output
- Below, on the picture, example:
 - `curl -i http://www.wp.pl/`
 - In response:
 - HTTP/1.1 301 Moved Permanently
 - There is a line in the header:
Location: <https://www.wp.pl/>
- Example on the next slide after the request to the secured server:
 - `curl -i https://www.wp.pl/`



[illegible]

Command `curl` – expanded header 2/2

- This time we get the answer (in the first line):
 - `HTTP/1.1 200 OK`
- Then there are multiple lines of headers from
 - `Server: nginx`
- Until:
 - `X-Op-Id-All: 5e2s`
- The next line is blank, so this is where the headers end.
- The response body, which is the body of the HTML page, begins on the next line, starting with the body line:
 - `<!DOCTYPE html>`
- Any subsequent blank lines will just be the content of the HTML document.

Command curl – other options

- With the option `-v` (verbose) you can see all the communication steps
 - It is best to redirect the result to a file (very many lines, below picture example):
`curl -i -v https://www.wp.pl > output.txt`
 - There will also be lines from the previous slide in the stream
- Selected options :
 - `-H, --header <header/@file>` Pass custom header(s) to server
 - `-G, --get` Put the post data in the URL and use GET
 - `-d, --data <data>` HTTP POST data

```
C:\Users\dariu\Documents>curl -i -v https://www.wp.pl > output.txt
* Rebuilt URL to: https://www.wp.pl/
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
0          0    0     0      0      0      0      0  0 --:--:-- --:--:-- --:--:--    0*   Trying 212.77.98.9...
* TCP_NODELAY set
0          0    0     0      0      0      0      0  0 --:--:-- --:--:-- --:--:--    0* Connected to www.wp.pl (212.77.98.9) port 443 (#0)
t 443 (#0)
* schannel: SSL/TLS connection with www.wp.pl port 443 (step 1/3)
* schannel: checking server certificate revocation
* schannel: sending initial handshake data: sending 180 bytes...
* schannel: sent initial handshake data: sent 180 bytes
* schannel: SSL/TLS connection with www.wp.pl port 443 (step 2/3)
* schannel: failed to receive handshake, need more data
* schannel: SSL/TLS connection with www.wp.pl port 443 (step 2/3)
* schannel: encrypted data got 3796
* schannel: encrypted data buffer: offset 3796 length 4096
* schannel: sending next handshake data: sending 93 bytes...
* schannel: SSL/TLS connection with www.wp.pl port 443 (step 2/3)
* schannel: encrypted data got 290
* schannel: encrypted data buffer: offset 290 length 4096
* schannel: SSL/TLS handshake complete
* schannel: SSL/TLS connection with www.wp.pl port 443 (step 3/3)
* schannel: stored credential handle in session cache
> GET / HTTP/1.1
Host: www.wp.pl
```

Communication process (1/4)

- The server gets **part** of the URI **after the server name**, e.g. `artykuly/20/url.html`
- Most often it receives an HTTP(S) GET request with the given name and protocol version :
`GET artykuly/20/url.html HTTP/1.1`
- Another common query is POST (when data from a **form** on a web page is sent). There are other kinds of requests, some of them will be about API controller programming.
- This line can be followed by header lines (created by the browser, normal user does not see this)
- For some types of requests, there may be an empty line and the request body (e.g. for POST there will be data from the form in the appropriate format)

Communication process (2/4)

- In response to the request, the server in the first line indicates the HTTP protocol version and describes the transaction status with a numeric code and a phrase:
 - For example, for success :
 - `HTTP/1.1 200 OK.`
 - And when it cannot locate the requested resource:
 - `HTTP/1.1 404 Not found`
- The server then sends out one or more HTTP headers, with additional information about the data being transferred.
 - The header name comes before the colon ':',
 - The components of the header values are separated by semicolons ';',
 - an example on slide 24

Communication process (3/4)

- For example, one of the HTTP headers for the example `www.wp.pl` is as follows:
 - `Content-type: text/html; charset=utf-8`
- This tells the browser that there will be a text file in the body (HTML5 code) and the utf-8 character encoding is used.
- The first part is the Multipurpose Internet Mail Extensions (MIME) type, invented for sending email but used ubiquitously :
 - <https://www.iana.org/assignments/media-types/media-types.xhtml>
- In this case, e.a. MIME type `text/plain` means that the information sent is text and can be displayed directly.
- Likewise, MIME type `image/jpeg` means the content is a JPEG image.
 - When the browser receives this MIME type, it tries to display the image.

Communication process (4/4)

- One blank line follows the headers
- Eventually, the server sends the content of the requested document (if there is no document name, it usually sends `index.html` by default) to `index.html`.
- The client-side browser then renders (or displays) the document, which may involve additional HTTP requests to get related CSS files, images, and other URIs discovered during rendering.
 - **Rendering** - graphical representation of digitally saved content in a form appropriate for a given environment (e.g. displaying a web page saved in HTML code in a browser window) (source: wikipedia.pl)

GET request - parameters in URL, query string

- The GET request may have **parameters** that follow the question mark '?',
`www.google.com/search?q=konieczny`
- In this case, 'search' is the name of the program/page that handles the forms on the Google server side, 'q' is the name of the variable in Google's search, 'konieczny' is the searched phrase.
- The string after the question mark is called **query string**.
- The 'name = value' **pair** is passed to the server and is separated by the equal sign '='.
- The pairs are **separated by** the ampersand '&' character.
- The data transmitted in this way is often used to find the appropriate resource on the server.
- The GET request has an **empty** request **body**.

POST request

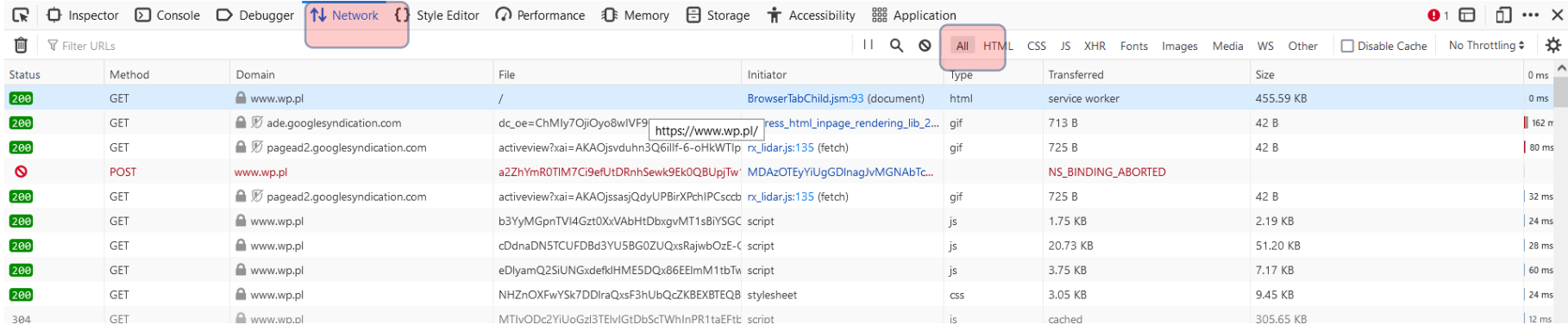
- The POST request sends **the form data** as the **body** of the HTTP message, **not** as **part** of the URL
 - By default, the format is the same as in the query string
 - There may be a different format, such as often JSON
- The GET request can be of some **limited length** which limits the length of the query string, so it is often necessary to send more information using the POST method.
 - Minimum URI length limit: **255 characters**
 - Limit per URI length (browser dependent): 2KB-8KB
 - Intermediaries on the way of the message and the server may add their own limitations (not less than 255)
- The POST method hides the transmitted data from the user, but for this reason it cannot be "linked" in another web document.
 - Linking can only be done using the URL, so it is not possible to enter anything into the body of the query being sent

Client-side cache, HEAD request

- Browsers often store the recently viewed web pages on the disk (**cache**) in order to load them quickly and to perform the "go back to the previous page" operation.
- If there are no changes between the cached version and the current version on the Internet, it **speeds up** browsing.
- The HEAD request only gets the response headers (no body).
- The HTTP response may indicate the amount of time (in the response header) that the content remains up-to-date.
- The browser can follow this data and load the document from the cache.
- There is also an HTTP response of "304 not modified", indicating that the file content has not changed since the last request.
- Browsers usually do not remember server responses to POST requests (e.g. with form data) because the next such request may not return the same response.
 - Additionally, it could generate a repeated operation (e.g. when we go back to the previous page) in the bank/shop, which is rather not expected by the user.

Browser development tools 1/3

- Switch in most browsers: <F12> key or <Ctrl>+<Shift>+<i>
- Sometimes page refreshes need to be performed (due to resource caching)
- Shows all requests made by the browser (also for photos, CSS, JS files etc.)
- Below is a screenshot for the Firefox browser for <https://www.wp.pl/>
- The tool window may be on the side of the window, in a different national language, etc., which will make it look a bit different.



The screenshot shows the Firefox Developer Tools Network tab. The 'Network' tab is selected, and the 'All' filter is active. The table below lists the network requests made by the browser.

Status	Method	Domain	File	Initiator	Type	Transferred	Size	Time
200	GET	www.wp.pl	/	BrowserTabChild.jsm:93 (document)	html	service worker	455.59 KB	0 ms
200	GET	ade.googleadsyndication.com	dc_oe=ChMly7OjOyo8wIVF9...	https://www.wp.pl/.../ress_html_inpage_rendering_lib_2...	gif	713 B	42 B	162 ms
200	GET	pagead2.googleadsyndication.com	activeview?xai=AKAOjsvduhn3Q6illf-6-oHkWTlp...	rx_lidar.js:135 (fetch)	gif	725 B	42 B	80 ms
404	POST	www.wp.pl	a2ZhYmR0TIM7C9efUtDRnhSewk9Ek0QBUpjTw...	MDAzOTeyYiUgGDInagJvMGNAbTc...		NS_BINDING_ABORTED		
200	GET	pagead2.googleadsyndication.com	activeview?xai=AKAOjsasjQdyUPBirXPchIPCscdb...	rx_lidar.js:135 (fetch)	gif	725 B	42 B	32 ms
200	GET	www.wp.pl	b3YyMGpnTVI4Gzt0XxVAbHtDbxgvMT1sBIYSGC...	script	js	1.75 KB	2.19 KB	24 ms
200	GET	www.wp.pl	cDdnaDN5TCUFDBd3YU58G0ZUQxsRajwbOzE-C...	script	js	20.73 KB	51.20 KB	28 ms
200	GET	www.wp.pl	eDlyamQ2SIUNGxdefklHME5DQx86EElmM1tbTw...	script	js	3.75 KB	7.17 KB	60 ms
200	GET	www.wp.pl	NHZN0XfWYSk7DDlraQxsF3hUbQcZKBEXBTEQB...	stylesheet	css	3.05 KB	9.45 KB	24 ms
304	GET	www.wp.pl	MTivODc2YUoGzI3TElvIGtDbScTWhInPR1taEft...	script	is	cached	305.65 KB	12 ms

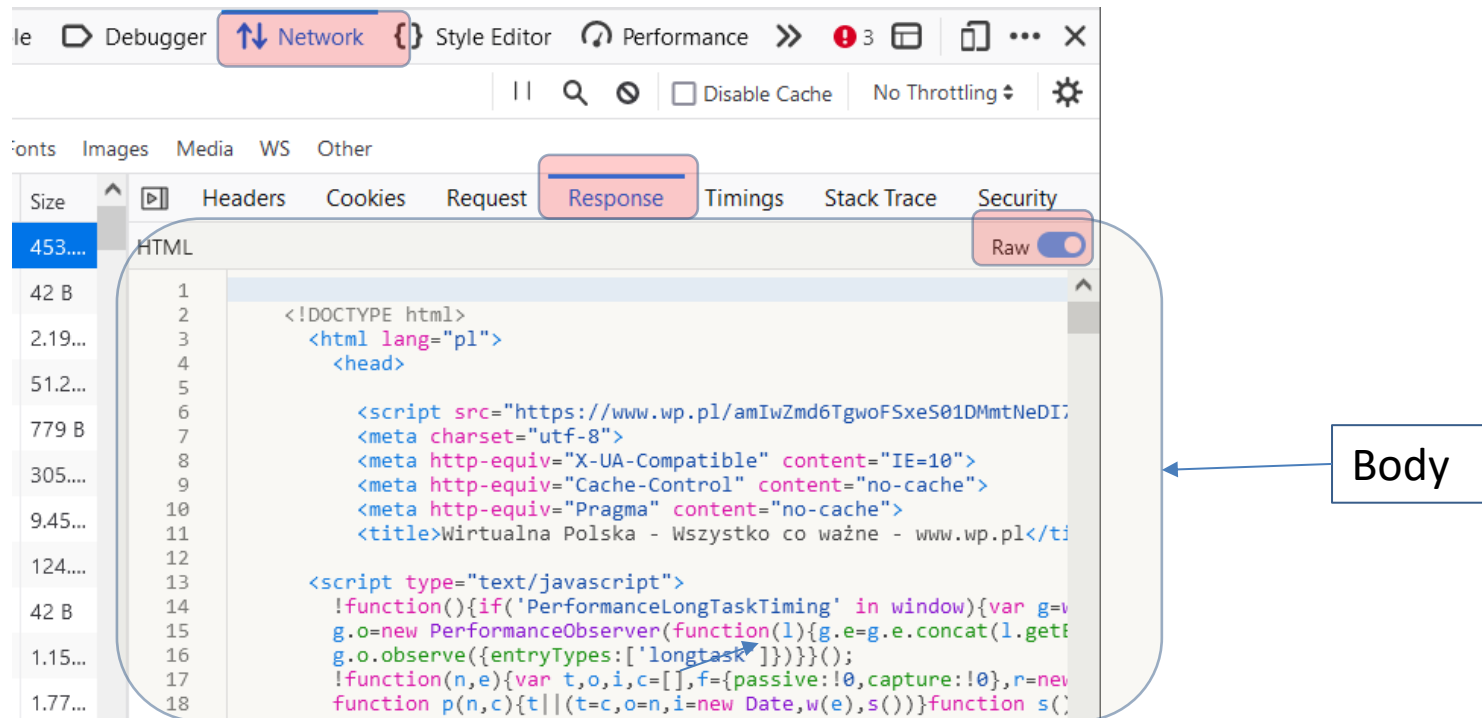
Browser development tools 2/3

- Below is a screenshot after clicking, for example, the first GET request `www.wp.pl` from the list
- In the "Headers" tab: Request, Response, Headers
- Latest in raw and processed versions.

[illegible]

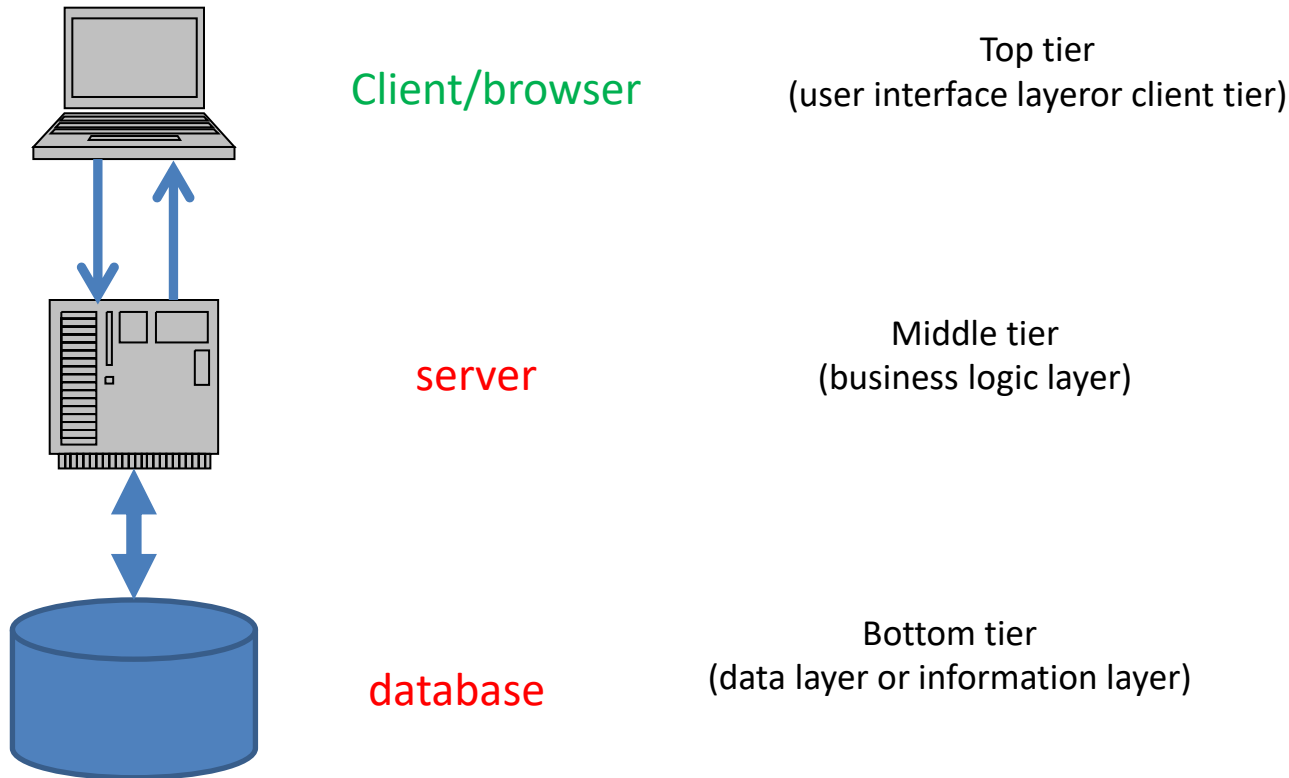
Browser development tools 3/3

- Below is a screenshot after clicking, for example, the first request from the list and in the "Response" tab: Response body, that is HTML document (also with raw switch).



Multi-tier application architecture 1/3

- Web applications are often multi-tier applications
- Layers are responsible for various logically related functionalities.
- Three-layer web application:
 - Client/browser – top tier, presentation
 - WWW Server – middle tier, business logic
 - Database – bottom tier, data



Multi-tier application architecture 2/3

- The **bottom layer** (**data** layer, **information** layer) manages the application data.
 - It stores, most often, data in a relational database management system (RDBMS).
- The **middle layer** (the **business logic** layer) controls the data needed for the interaction between the application's clients and its data.
 - The middle tier acts as an intermediary between the data in the information tier and the application clients.
 - The middle tier control logic processes client requests and recovers data from the database.
 - The part of the middle layer responsible for the presentation logic processes the data from the information layer and presents the content to the client.
- Web applications usually present data to the client as HTML documents.

Multi-tier application architecture 3/3

- The business logic in the middle tier watches the business rules and ensures that the data is trustworthy before the application updates the database or presents the data to the user.
- Business rules determine how clients [applications, not humans] access data and how applications process the data.
- The **top** tier (**client** tier) contains the application's graphical interface that collects input and displays the output.
- The user controls the application through the user interface, which is usually a web browser or a mobile device.
- The user takes an action (e.g. clicking a hyperlink), in response, the client layer contacts the middle layer, which he sends a request to (through the middle layer) recover data from the information layer. After obtaining them, it displays the data obtained for the user.

Client-side scripts

- Client-side scripts in JavaScript can be used:
 - to validate the input data entered by the user,
 - to extend the functionality of websites, their graphical attractiveness
 - to support "self-contained" communication between the browser and the web server.
 - for fixing collisions between frameworks
- Client-side scripting limitations:
 - browser dependency - browser or so-called the scripting host **must support** the language and its capabilities.
 - for security reasons they have **limited access** to local **hardware** and **file system**.
 - the source code of the client-side scripts **can be viewed** by the client ("show page source" option and development tools).
 - Confidential information (database passwords, user passwords, personal data) should **not be** on the client's side.
 - Although the client will check the data, e.g. in the form, this checking must also be duplicated on the server side.
 - For example, you can use `curl` to query the middle tier without the client tier
 - putting certain operations into client-side JavaScript can also be dangerous for the entire application.
- In fact, the only scripting language is JavaScript.

Server-sides scripts/programs

- Ability to use a large variety of programming languages (theoretically any mature language that has the ability to stream and register with the services of a given operating system or network service server).
 - PHP
 - Java
 - C #
 - e.t.c.
- Many ready-made servers with the possibility of extending the functionality:
 - MS IIS - ASAPI extensions
 - Apache – modules
 - e.t.c.
- Extensions can be written in different languages
- Server-side scripts / programs can communicate with other servers to download information, process it and send the result back to the client (comparison engine)
 - Among other things, they communicate with the data layer
- Most of the time, scripts need to run for multiple sessions
 - The solution should be scalable (if the number of requests increases, it is enough to buy hardware, install the software and the response time will not increase significantly).

Basics of the language

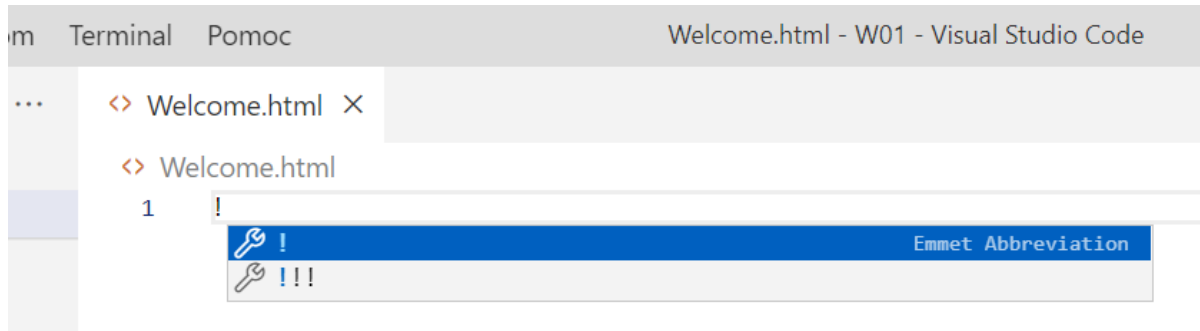
HTML5

Introduction

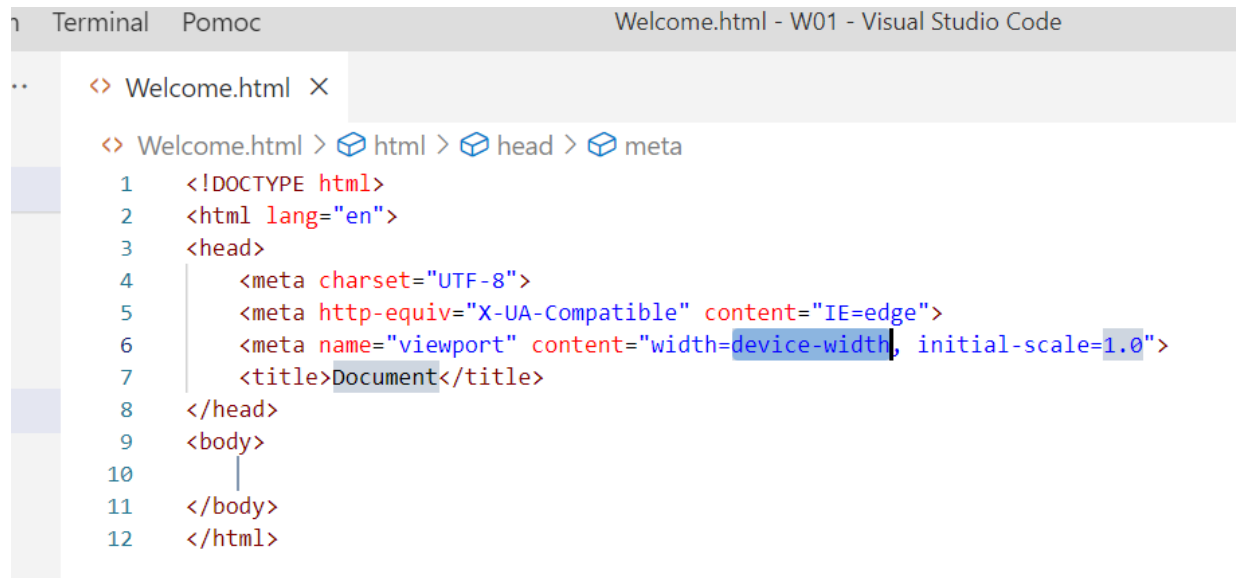
- **HTML5 (HyperText Markup Language 5)** - a markup language that describes **the structure** and **content** of documents displayed in web browsers
- Easiest creation - any text editor:
 - Notebook
 - Notepad ++
 - vi/vim (in linux)
 - Visual Studio Code with relevant plugins (will mainly be used in lecture examples)
- Save the file with the `.html` extension (once also `.htm`)
- Then open the file in the browser (e.g. by double-clicking on it)

Creating an HTML scaffold in VS Code

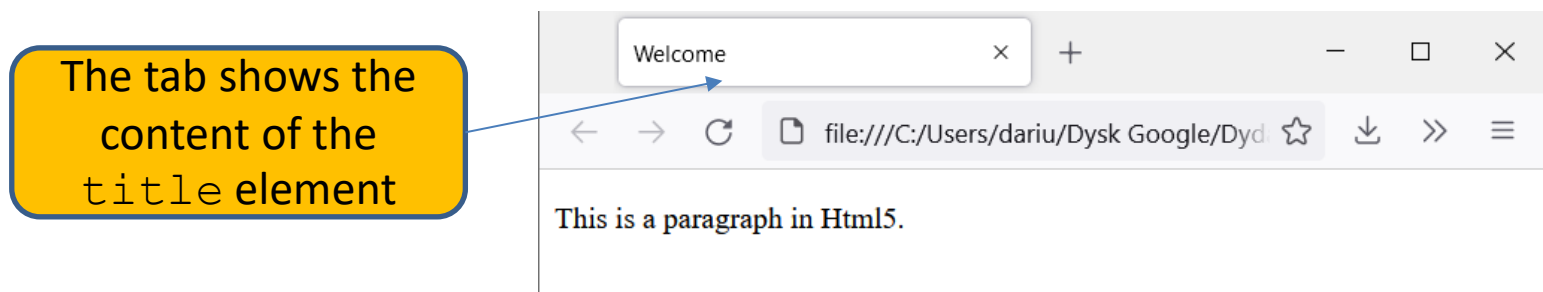
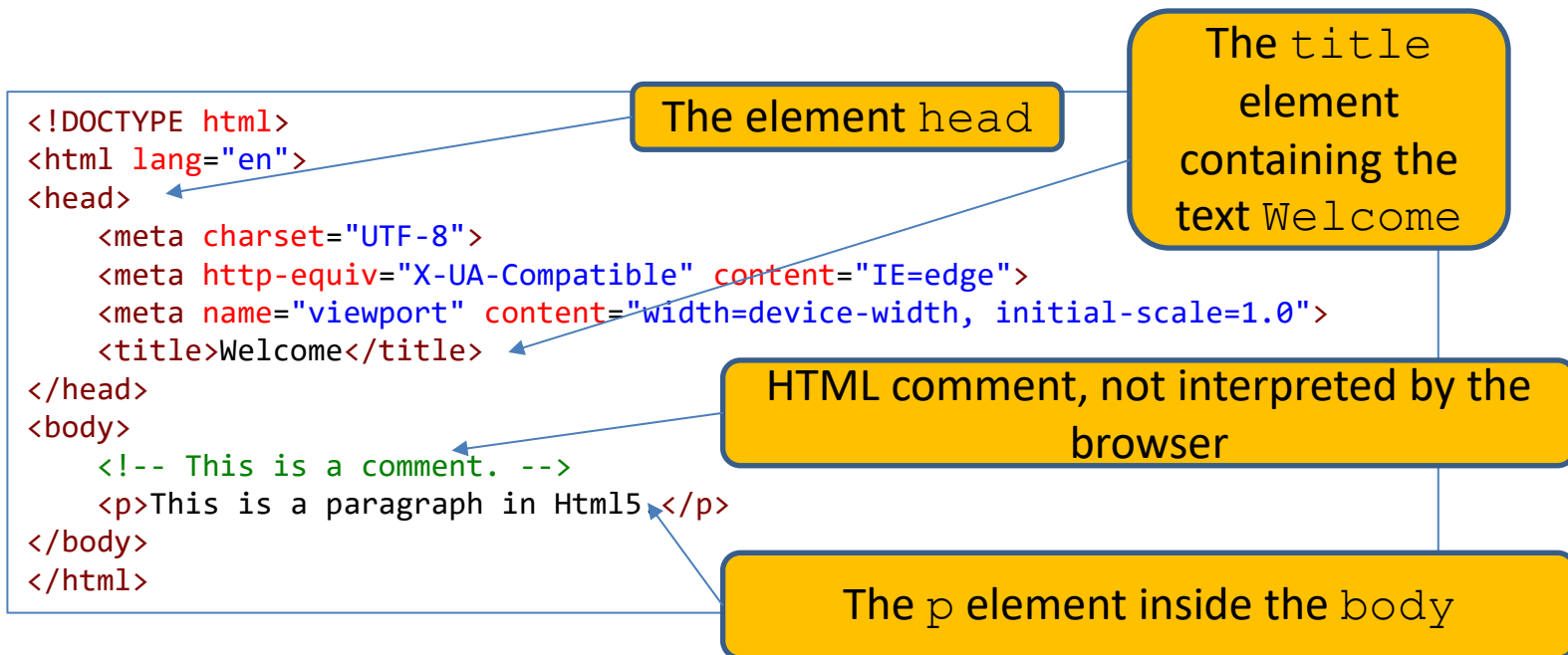
- After creating and opening the `Welcome.html` file in VS Code, enter the **exclamation mark** (`!`):



- And then the `<tab>` key. It will work the so-called **emmet** (for files with the `.html` extension) and will create the skeleton of the HTML page



Sample welcome.html file and opening in browser



Sample File Components (1/3)

- Document Type Declaration (DOCTYPE)
 - required in HTML5 documents for browsers to render the page according to the latest standards.
 - When there is no declaration - some browsers operate in a special mode ("quirks mode").
- Comments:
 - begin with `<!--`
 - end with `-->`
 - they can be multi-line

Sample File Components (2/3)

- HTML5 documents delimit most elements with a start and end tag.
 - The **start tag** consists of the item name in angle brackets
 - For example, `<html>`
 - The **end tag** consists of the element name preceded by a slash (/) in angle brackets
 - For example, `</html>`
- There are so-called "Empty elements" that have **no end tags**.
 - E.a.: `<meta>`
 - A side note: XHTML derived from XML must have tag closure, so HTML should use `<br>`, and XHTML `<br> </br>` or one notation `<br/>`.
 - Most browsers will correctly fix the use of XHTML syntax in HTML.
- Many start tags have attributes that convey additional information about the element. Browsers use them to determine how to process an item.
 - An attribute has a name and a value separated by an equal sign (=).
 - Values **must be** in quotation marks.
 - Attributes are separated by whitespace (space, tab, Enter)
 - E.a.:
 - `<html lang="en">`
 - `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

Sample File Components (3/3)

- The `<head>` element
 - called a nested element because it is enclosed between the start and end tags of the `<html>` element.
- The `<title>` element
 - it is also a nested element because it is enclosed between the start and end tags of the `<head>` element.
 - describes the website.
 - the titles usually appear:
 - in the title bar at the top of the browser window,
 - in the browser tab on which the page is displayed,
 - as text that identifies the page when the user adds it to the favorites list.
 - Search engines use the title for indexing and displaying results.
- Element of paragraph `<p>`
 - All text between `<p>` and `</p>` is one paragraph.

Validators HTML5

- HTML5 documents that are syntactically correct will be displayed correctly.
- HTML5 documents with syntax errors may not display correctly.
 - Browsers try to guess where the error is (e.g. closing the tag) and finally display the page, but each of them can fix the errors differently
- It is worth using validating services (e.g. `validator.w3.org/#validate-by-upload`) or **plugins** and development environments that have built-in validators.

Headers

- HTML5 offers six headers (from `<h1>` to `<h6>`) for determining the relative importance of the information
- The `<h1>` header element is considered **the most** relevant header and is rendered with the **largest** font.
- Each subsequent header element (i.e. `<h2>`, `<h3>`, etc.) is rendered in **smaller** and smaller fonts
- This **can be changed** with attributes or CSS, but the above relationship should be maintained.
- Hint for VS Code: entering the string `"lorem"` `<TAB>` we get a longer text in Latin, which serves as a good filler words

Headers from <h1> to <h6> - example of use (Headers.html)

Containing tags for the cursor position

<> Headers.html ✕

<> Headers.html > html > body > p

```
1 <!DOCTYPE html>
2 <!-- Header from h1 to h6-->
3 <html>
4 <head>
5   <meta charset="UTF-8">
6   <title>Headers</title>
7 </head>
8 <body>
9   <h1>Header h1</h1>
10  <p>Lorem ipsum dolor sit amet consectetur, adipisicing
11  <h2>Header h2</h2>
12  <p>Lorem ipsum dolor sit amet consectetur, adipisicing
13  <h3>Header h3</h3>
14  <p>Lorem ipsum dolor sit amet consectetur, adipisicing
15  <h4>Header h4</h4>
16  <p>Lorem ipsum dolor sit amet consectetur, adipisicing
17  <h5>Header h5</h5>
18  <p>Lorem ipsum dolor sit amet consectetur, adipisicing
19  <h6>Header h6</h6>
20  <p>Lorem ipsum dolor sit amet consectetur, adipisicing
21 </body>
22 </html>
```

Cursor

Header h1

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Ullam similique rem, repellat consectetur vel, mollitia qui provident facilis voluptatem veritatis possimus natus impedit nisi nihil molestias voluptatum ipsa sapiente. Quibusdam.

Header h2

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Ullam similique rem, repellat consectetur vel, mollitia qui provident facilis voluptatem veritatis possimus natus impedit nisi nihil molestias voluptatum ipsa sapiente. Quibusdam.

Header h3

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Ullam similique rem, repellat consectetur vel, mollitia qui provident facilis voluptatem veritatis possimus natus impedit nisi nihil molestias voluptatum ipsa sapiente. Quibusdam.

Header h4

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Ullam similique rem, repellat consectetur vel, mollitia qui provident facilis voluptatem veritatis possimus natus impedit nisi nihil molestias voluptatum ipsa sapiente. Quibusdam.

Header h5

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Ullam similique rem, repellat consectetur vel, mollitia qui provident facilis voluptatem veritatis possimus natus impedit nisi nihil molestias voluptatum ipsa sapiente. Quibusdam.

Header h6

Lorem ipsum dolor sit amet consectetur, adipisicing elit. Ullam similique rem, repellat consectetur vel, mollitia qui provident facilis voluptatem veritatis possimus natus impedit nisi nihil molestias voluptatum ipsa sapiente. Quibusdam.

Links - <a> tag

- Hyperlinks refer or link to other resources, such as HTML5 documents or images.
 - Web browsers usually underline the text of the hyperlink and color it blue (if it has not been visited yet) or purple (if it has been visited).
- The <a> (anchor) tag usually contains the `href` (hypertext reference) attribute, which **specifies the location** of the resource e.a.
 - website or website location
 - File
 - e-mail address
- **The inside** of the tag is the **text** that **will appear** in your browser and **can be clicked** to launch the **link**.
- When the URL in this attribute does not point to any specific document on the page, the web server returns the default page. Typically this is `index.html` (or `index.php`, `index.asp`), however the web server can be configured to return a different file/stream as the default page.
- Anchors can link **to an email address**
 - by `<a href= "mailto:mailaddress-message">... </a>`
 - Clicking such a link will **launch** the **default e-mail client** with the field of the e-mail recipient filled in.
- If we want the **new page** to open in a **new tab**, add the `target = "_ blank"` attribute.
- Other anchor options will follow later.

Links – example use (Links.html)

```
<!DOCTYPE html>
<!-- Links -->
<html>
<head>
  <meta charset="UTF-8">
  <title>Links</title>
</head>
<body>
  <h1>My Links</h1>
  <p>Interesting links for garden:</p>
  <p><a href="https://zielonyogrodek.pl/">Zielony ogródek (Green garden)</a></p>
  <p><a href="https://fajnyogrod.pl/">Fajny ogródek (Nice garden)</a></p>
  <p><a href="https://poradnikogrodnicy.pl/index.php">Poradnik ogrodnicy (A gardening guide)</a></p>
  <p>author: <a href="mailto:dariusz.konieczny@pwr.edu.pl">Dariusz Konieczny</a></p>
</body>
</html>
```

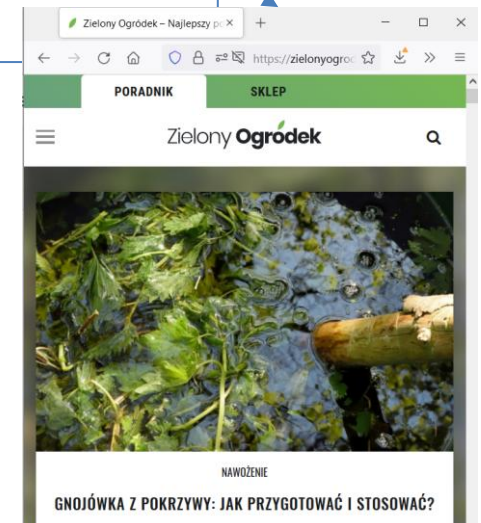
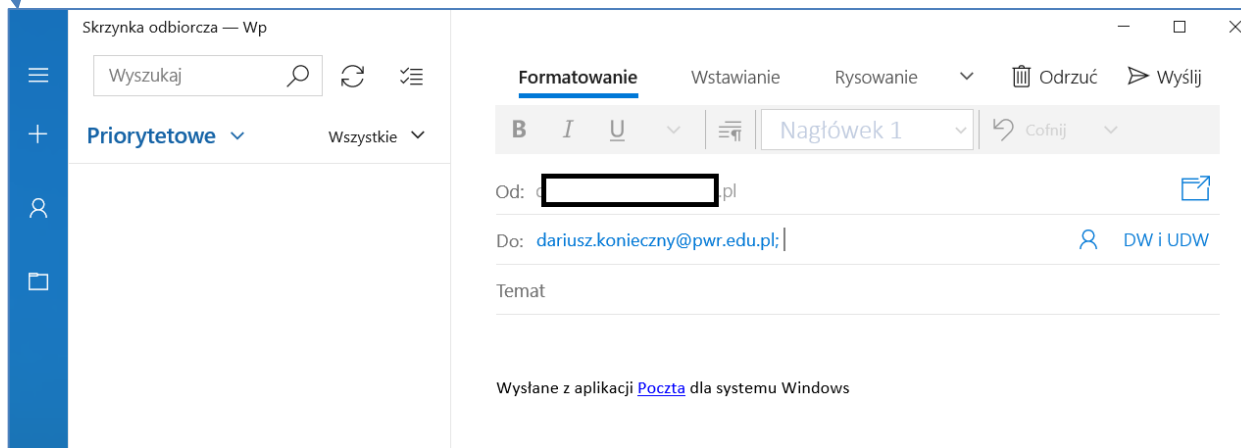
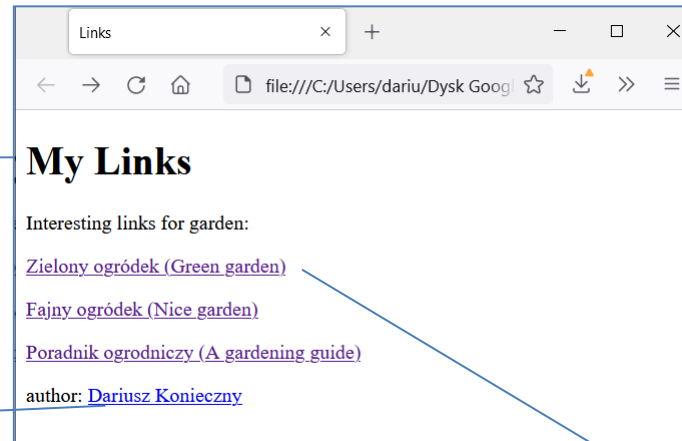


Image - `<img>` tag

- The most popular image format used by web designers today is PNG (Portable Network Graphics), JPEG (Joint Photographic Experts Group), GIF (Graphics Interchange Format). Depending on the browser, other graphic formats are or are not supported.
- `<img>` tag - to insert a picture
 - It is an "empty element", there is **no closing tag**
 - The `src` attribute of the `<img>` element specifies the location of the image
 - The `width` and `height` attributes are optional, they specify the width and height of the image in pixels
 - If they are omitted, the browser will use the actual width and height of the image
 - The `title` attribute specifies the text that will appear when you hover your mouse over the image.

Example of - code Images.html

```
<!DOCTYPE html>
<!-- Images-->
<html>
<head>
  <meta charset="UTF-8">
  <title>Images</title>
</head>
<body>
  <h1>My images</h1>
  <p> incorrect ratio:
    
    correct ratio:
    
    another image:
    
    non-existent image:
    
  </p>
  <p>
    outside source:
    
  </p>
  <p>
    Original size:
  </p>
  <p>
    
  </p>
</body>
</html>
```

Example of `<img>` - effect

Images

file:///C:/Users/dariu/Dysk Google/Dydaktyka/2021-2022/semestr zimowy 2021_22/Aplikacje webowe na platformę .NET/Wykł

My images

incorrect ratio:  correct ratio:  another image:  non-existent image: rose's flowers

outside source: **allegro**

Original size:



Hints to the `<img>` element

- It is advisable to always use the `width` and `height` attributes in the `<img>` element so that the browser knows **how much** screen **space** is allocated to the image when it loads the HTML5 file. This **allows** the browser to **arrange** the rest of the elements on the page correctly, even **before the image loads**. This will allow you to **speed up** the **rendering** and **display** of the page.
- **Typing** image **sizes** that **alter** its **proportions** (width to height) often **degrades** image quality. It's best to use the same (or similar) width-to-height ratios for these attributes relative to the original dimensions of the image.
- Each `<img>` element in an HTML5 document should have an `alt` attribute.
- Browser may not be able to render the image (temporarily unavailable, incorrect format). Then it will display the value of the `alt` attribute.
- The `alt` attribute is also important for accessibility - speech synthesizers for the blind read the value of the `alt` attribute. For this reason, the `alt` attribute should describe the content of the image with essential details for understanding the content.

Picture as link

- By inserting an image inside the `<a>` element, we will get a **clickable image** linking to the selected place.
 - In addition to the picture, text can be used at the same time
 - If only an image is linked as a link, the `alt` attribute should be set to ensure accessibility for people with disabilities.
Otherwise, it is not necessary (or even rather **inadvisable**).

Picture as link – example use (ImagesAsLinks.html)

```
<!DOCTYPE html>
<!-- Images as links-->
<html>
<head>
  <meta charset="UTF-8">
  <title>Images as links</title>
</head>
<body>
  <h1>Links with images</h1>
  <p>Only an image as a link to allegro website:
    <a href="https://allegro.pl/">
      </a>
      <!-- </a> in this line will create clickable SPACE character. -->
    A text after the anchor.
  </p>
  <p>An image and a text as a link to allegro website:
    <a href="https://allegro.pl/">
      
      jump to Allegro.</a>
    A text after the anchor.
  </p>
</body>
</html>
```

